ARRAYS

Coders.Eleven - JAVA

Coders.Eleven JAVA
Arrays

#1 Eindimensionale Arrays

Motivation

Deklarieren & Initialisieren

Index

For-Each-Schleife

#2 Referenzen

Zuweisung

Speicher

Kopieren

Vergleichen

#3 Hilfsklasse Arrays



#1 Eindimensionale Arrays

Motivation

Deklarieren & Initialisieren

Index

For-Each-Schleife



Motivation

- Bisher konnten wir mit einer Variable genau einen Wert speichern
- Wie speichern wir jedoch am effizientesten zB. 1000 Werte?
- Wir brauchen also eine Struktur, die das Verwalten vieler Werte erleichtert



Ein Array ist eine Datenstruktur, welche gleichartige Elemente zusammenfasst.

Datenstruktur

- Legen größere Menge an Daten strukturiert ab
- Unterstützen den gezielten Zugriff auf die Daten

Arrays deklarieren

- Form: `Datentyp[] name;`
- Achtung:
  - Es wird noch keine Größe festgelegt
  - Legt nur fest, dass zB. `numbers` auf ein Array von ganzen Zahlen zeigt

```
int[] numbers;

double[] temperature;

String[] names;
```

Arrays anlegen

```
int[] numbers = new int[5];
```

Anlegen bei Deklaration

new-Operator:
- Neuer Speicher wird angefordert
- Ein Array-Objekt der richtigen Größe wird erzeugt

```
int[] numbers;

//...

numbers = new int[5];
```

Späteres Anlegen

```
int[] numbers = new int[] {1, 2, 3, 4, 5};
```

```
int[] numbers = {1, 2, 3, 4, 5};
```

Anlegen mit eigener Initialisierung

Index

- Jedem Element eines Arrays ist ein Index vom Typ `int` zugewiesen
- Achtung:
  - Indexzählung beginnt bei 0
  - Indexzählung geht bis (Länge des Arrays) – 1
- Größe des Arrays wird (in unserem Fall) mit `numbers.length` abgefragt

```java
int[] numbers = new int[5];
```

Index	**0**	**1**	**2**	**3**	**4**
Inhalt	0	0	0	0	0

Zugriff auf einzelne Elemente

- Indexwert in eckigen Klammern
- Kann wie eine Variable verwendet werden
- Index kann ein beliebiger Ausdruck vom Typ `int` (bzw. `short`, `byte` oder `char`) sein

```
int[] arr = new int[5];
int n = arr.length;

arr[0] = 2;
arr[1] = arr[0] + 3;
arr[2]++;
arr[3] = arr[n-3];
arr[4] = arr[arr[2]];

for(int i = 0; i < n; i++) {
    System.out.println(arr[i]);
}
```

```
2
5
1
1
5
```

For-Each Schleife

- Alle Elemente des Arrays werden durchlaufen
- Elementvariable (im Beispiel `i`) erhält den Wert des nächsten Array-Elements
- Schleifenrumpf wird für jedes Element des Arrays ausgeführt

```
for (int i : arr) {
    System.out.println(i);
}
```

```
2
5
1
1
5
```

#2 Referenzen

Zuweisung

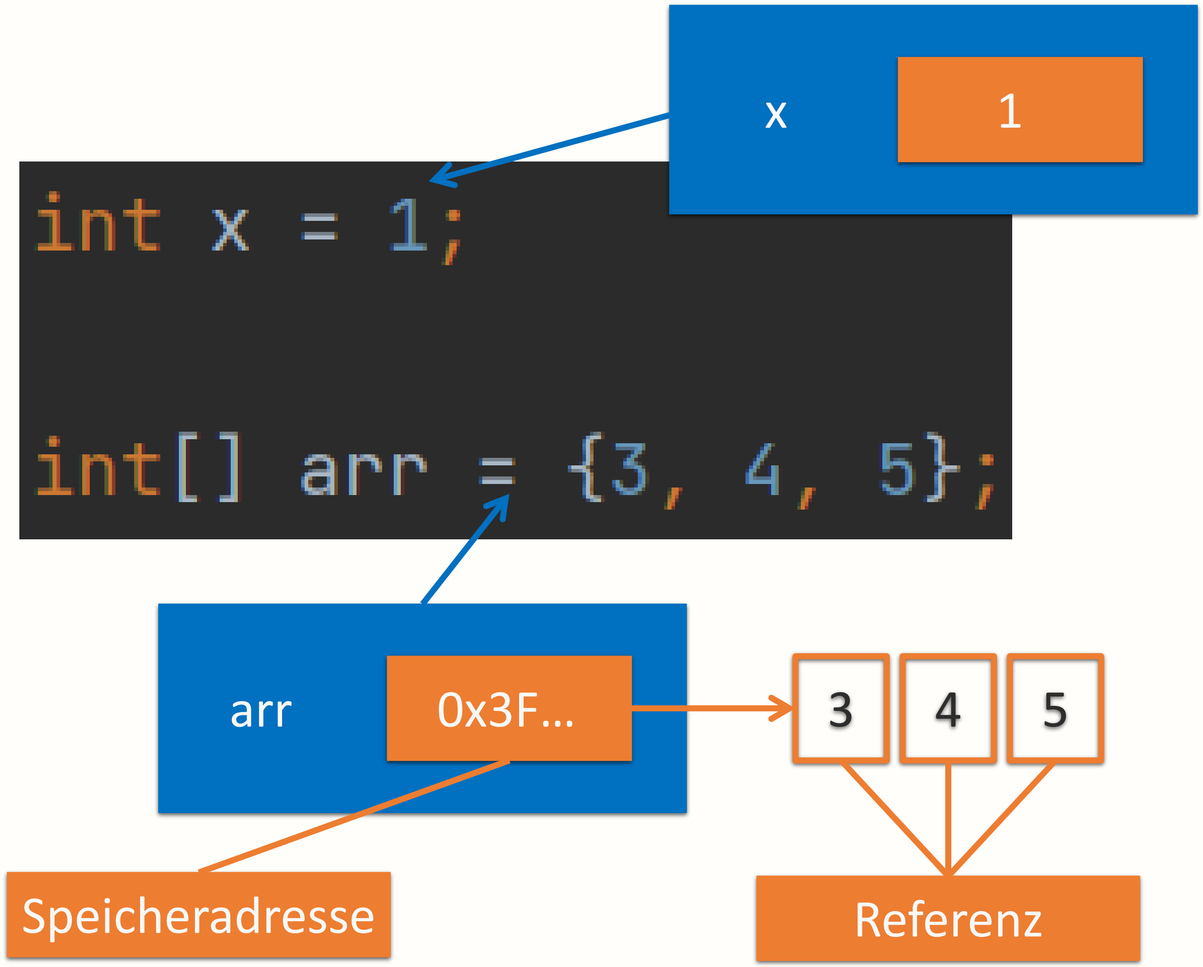
Speicher

Kopieren

Vergleichen



Referenztypen



Zuweisung bei Array-Variablen

- Wie lautet die Ausgabe?
- x und y sind Array-Variablen
- x und y zeigen auf den gleichen Speicherbereich
  - x und y sind „Aliase“
- Zuweisung bei `y[1]` verändert auch `x[1]`

```
int[] x = new int[10], y;

y = x;
y[1] = 7;

System.out.println(y[0] + " " + x[0]);
System.out.println(y[1] + " " + x[1]);
```

```
0 0
7 7
```

Freigabe des Array-Speichers

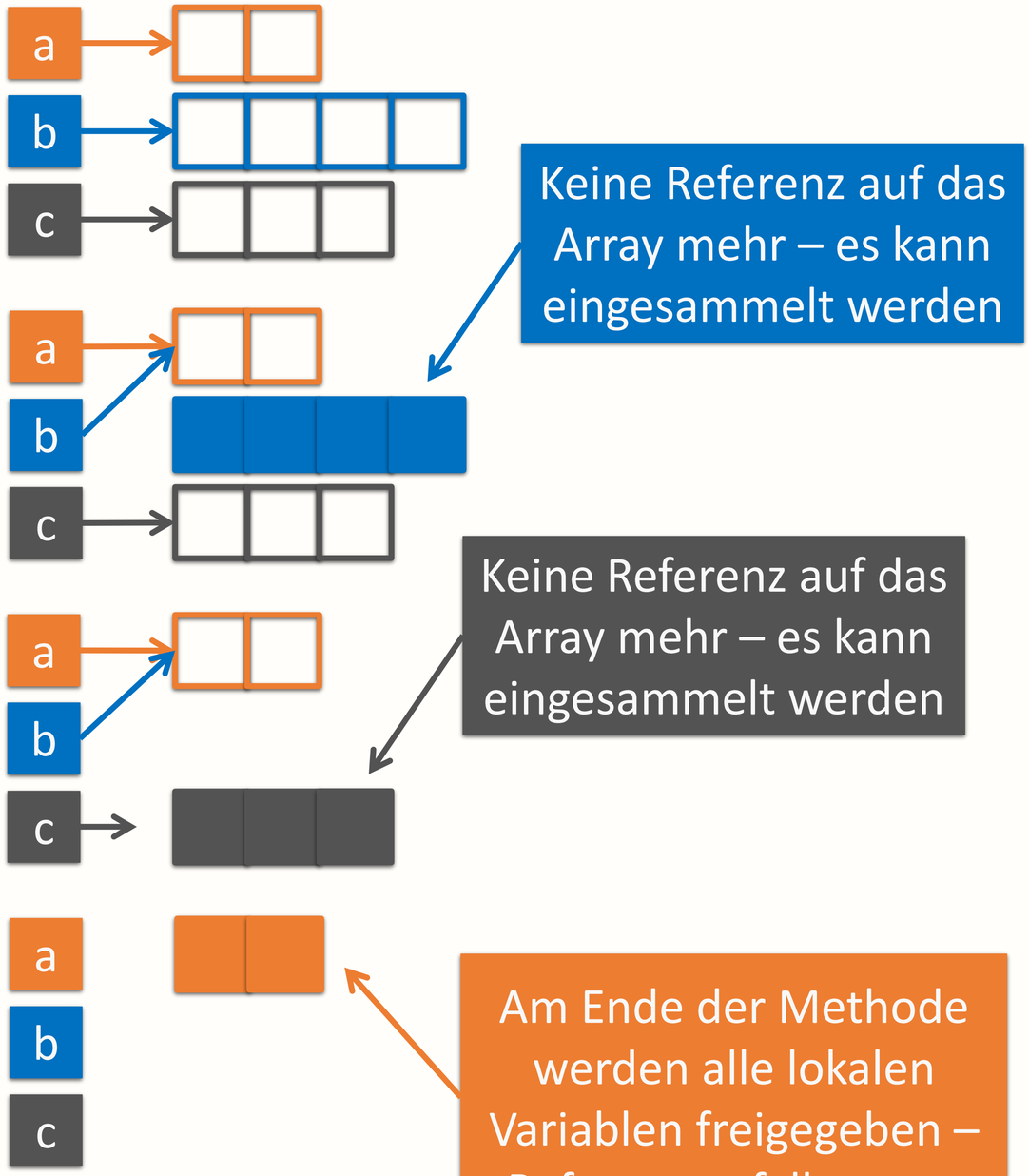
- Erfolgt automatisch durch das System
  - Speicher muss nicht explizit freigegeben werden
  - Nicht benutzter Speicher wird zur Laufzeit automatisch eingesammelt (Garbage Collector)
- Voraussetzung für das Einsammeln:
  - Array wird nicht mehr referenziert („Nichts zeigt mehr darauf“)
- Erreichbarkeit unterbinden
  - Einer Array-Variable ein anderes Array zuweisen
  - Einer Array-Variable den Wert `null` zuweisen

Beispiel

```
int[] a = new int[2];
int[] b = new int[4];
int[] c = new int[3];

b = a;

c = null;
```



Kopieren von Arrays

- Echte Kopie erzeugen durch elementweises Kopieren (siehe rechts oben)
- Methode `System.arraycopy` (siehe unten)
  - Effiziente Methode
  - Kopieren von Abschnitten bzw. ganzen Arrays

```
int[] original = {3, 4, 5, 6, 7};
int[] copy = new int[original.length];

for(int i = 0; i < original.length; i++) {
    copy[i] = original[i];
}
```

```
System.arraycopy(original, srcPos: 0, copy, destPos: 0, original.length);
```

Identität und Gleichheit

- Vergleichsoperator == bei Arrays nicht geeignet (gilt auch für `!=`)
  - Vergleicht nur, ob Arrays die gleichen Referenzen haben
  - Vergleicht nicht den Inhalt der Arrays
- Identität
  - Zwei Werte bzw. Objekte sind identisch, wenn die Anwendung von „==“ `true` liefert
- Gleichheit
  - Zwei Objekte sind gleich, wenn ihr Inhalt übereinstimmt (`equals()` liefert `true`)
- Unterscheidung
  - Gleiche Werte von elementaren Typen (zB. `int`) sind immer identisch
  - Identische Objekte (zB. Arrays) sind stets gleich
  - Gleiche Objekte (zB. Arrays mit gleichem Inhalt) müssen aber nicht identisch sein

#3 Hilfsklasse Arrays



Nützliche Methoden

- Hilfsklasse für Arrays muss importiert werden (`import java.util.Arrays;`)
- Nützliche Methoden (hier für `int`):
  - `static int binarySearch(int[] a, int key)`
    - Binäre Suche in einem Array
  - `static boolean equals(int[] a, int[] a2)`
    - Vergleicht den Inhalt von zwei Arrays
  - `static void fill(int[] a, int val)`
    - Befüllt alle Stellen eines Arrays mit einem bestimmten Wert
  - `static void sort(int[] a)`
    - Sortiert den Inhalt eines Arrays aufsteigend
  - `static String toString(int[] a)`
    - Liefert den Inhalt eines Arrays als formatierten String zurück